

## TP 1

### Partie 0 : Préparation de l'environnement de développement Java

Avant de commencer les exercices, chaque étudiant doit disposer d'un poste capable de **compiler** et d'**exécuter** un programme Java. Suivez les étapes ci-dessous dans l'ordre, puis validez votre installation à l'étape 0.6.

#### 0.1 Rappel : JDK, JRE et JVM

| Sigle      | Signification            | Rôle                                                                                                                                                              |
|------------|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>JVM</b> | Java Virtual Machine     | Exécute le bytecode (fichiers <code>.class</code> ) ; c'est elle qui rend Java portable.                                                                          |
| <b>JRE</b> | Java Runtime Environment | JVM + bibliothèques standards : suffit pour exécuter un programme.                                                                                                |
| <b>JDK</b> | Java Development Kit     | JRE + outils de développement ( <code>javac</code> , <code>java</code> , <code>jar</code> , <code>javadoc</code> ...) : <b>c'est le JDK qu'il faut installer.</b> |

#### 0.2 Installation du JDK

Installez une version **LTS** (Long-Term Support) du JDK : **JDK 25** est recommandé (JDK 21 convient également). Deux distributions gratuites sont possibles :

- **Oracle JDK** : [oracle.com/java/technologies/downloads](https://oracle.com/java/technologies/downloads)
- **Eclipse Temurin (OpenJDK)** : [adoptium.net](https://adoptium.net)

| Système                        | Procédure                                                                                                                                                                                                           |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Windows</b>                 | Télécharger l'installateur <b>x64 MSI Installer</b> , l'exécuter et conserver le dossier proposé (par défaut <code>C:\Program Files\Java\jdk-25</code> ). Avec Temurin, cocher l'option « Set JAVA_HOME variable ». |
| <b>Linux (Ubuntu / Debian)</b> | Dans un terminal : <code>sudo apt update</code> puis <code>sudo apt install openjdk-25-jdk</code> (ou <code>openjdk-21-jdk</code> si la version 25 n'est pas proposée par votre distribution).                      |
| <b>macOS</b>                   | Télécharger le paquet <code>.pkg</code> (aarch64 pour les puces Apple, x64 pour Intel) et l'installer, ou avec Homebrew : <code>brew install --cask temurin</code> .                                                |

#### 0.3 Vérification de l'installation

Ouvrez un **nouveau** terminal (Invite de commandes ou PowerShell sous Windows) et tapez les deux commandes suivantes. Chacune doit afficher un numéro de version ; les deux versions doivent être identiques.

```
java -version
javac -version
```

#### 0.4 Variables d'environnement (si une commande n'est pas reconnue)

**Sous Windows** : Menu Démarrer → « Modifier les variables d'environnement système » → bouton « Variables d'environnement... ».

1. Dans « Variables système », cliquer sur « Nouvelle... » : nom `JAVA_HOME`, valeur = dossier d'installation du JDK (exemple : `C:\Program Files\Java\jdk-25`).
2. Sélectionner la variable `Path` → « Modifier... » → « Nouveau » → ajouter `%JAVA_HOME%\bin`.

3. Valider, **fermer puis rouvrir** le terminal, et vérifier avec `echo %JAVA_HOME%` puis `javac -version`.

**Sous Linux / macOS :** ajouter les deux lignes suivantes à la fin du fichier `~/ .bashrc` (ou `~/ .zshrc`), en adaptant le chemin, puis rouvrir le terminal.

```
export JAVA_HOME=/usr/lib/jvm/java-25-openjdk-amd64
export PATH=$JAVA_HOME/bin:$PATH
```

### 0.5 Installation d'un environnement de développement (IDE)

Choisissez **un** des IDE suivants : **IntelliJ IDEA** ([jetbrains.com/idea](https://jetbrains.com/idea)), **Eclipse IDE for Java Developers** ([eclipse.org/downloads](https://eclipse.org/downloads)) ou **Visual Studio Code** avec l'extension « Extension Pack for Java ». Création d'un projet (exemple avec IntelliJ IDEA) :

1. *New Project* → Language : *Java* → Build system : *IntelliJ* → JDK : sélectionner le JDK installé → nommer le projet TP1 → *Create*.
2. Clic droit sur le dossier `src` → *New* → *Java Class* → saisir le nom de la classe.
3. Exécuter avec le triangle vert situé à gauche de la méthode `main` (ou *Run* → *Run...*).

### 0.6 Premier programme : validation de l'installation

Créez un fichier nommé `Bonjour.java` contenant le code suivant. **Rappel** : le nom du fichier doit être exactement celui de la classe publique, majuscules comprises.

```
public class Bonjour {
    public static void main(String[] args) {
        System.out.println("Bonjour, Java est bien installé !");
    }
}
```

Compilez puis exécutez-le **en ligne de commande**, depuis le dossier qui contient le fichier :

```
javac Bonjour.java      (compilation : produit le fichier Bonjour.class)
java Bonjour            (exécution : sans l'extension .class)
```

**Travail demandé :** (a) relevez la version affichée par `java -version` ; (b) exécutez `Bonjour` une fois en ligne de commande et une fois depuis l'IDE ; (c) expliquez en une phrase la différence entre les commandes `javac` et `java`.

### 0.7 Erreurs fréquentes

| Message                                                                                       | Cause et solution                                                                                                                                    |
|-----------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| « javac n'est pas reconnu en tant que commande interne... »                                   | Le dossier <code>bin</code> du JDK n'est pas dans le <code>Path</code> (ou seul un JRE est installé) : reprendre l'étape 0.4 et rouvrir le terminal. |
| <code>class Bonjour is public, should be declared in a file named Bonjour.java</code>         | Le nom du fichier ne correspond pas au nom de la classe publique : renommer le fichier.                                                              |
| <code>Could not find or load main class Bonjour</code>                                        | Commande lancée depuis un autre dossier, ou <code>java Bonjour.class</code> au lieu de <code>java Bonjour</code> .                                   |
| <code>java -version</code> et <code>javac -version</code> affichent deux versions différentes | Plusieurs installations de Java coexistent : placer <code>%JAVA_HOME%\bin</code> en première position dans le <code>Path</code> .                    |

## Partie 1 : Initiation à JAVA

### 1.1 Principes à retenir

| Principe                      | À retenir                                                                                                                                                                        |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Typage statique</b>        | Toute variable a un type déclaré : <code>int</code> , <code>double</code> , <code>boolean</code> , <code>char</code> (types primitifs) ou une classe comme <code>String</code> . |
| <b>Conversion</b>             | ( <code>int</code> ) <code>12.5</code> vaut <code>12</code> : la conversion d'un <code>double</code> en <code>int</code> supprime la partie décimale.                            |
| <b>Structures de contrôle</b> | <code>if / else</code> , <code>switch</code> , et les boucles <code>for</code> , <code>while</code> et <code>do ... while</code> .                                               |
| <b>Méthodes</b>               | Un traitement nommé, avec des paramètres et un type de retour ; <code>static</code> permet de l'appeler sans créer d'objet.                                                      |
| <b>Tableaux</b>               | Taille fixée à la création ; indices de <code>0</code> à <code>length - 1</code> .                                                                                               |
| <b>Chaînes</b>                | <code>String</code> est un objet : on compare le contenu avec <code>equals()</code> , jamais avec <code>==</code> .                                                              |

### 1.2 Exercice : le carnet de notes

Écrire un programme `CarnetDeNotes` qui demande le nombre de notes d'un étudiant, puis lit ces notes au clavier, une par une. Une note doit être comprise entre 0 et 20 : toute autre valeur est refusée et redemandée. Le programme affiche ensuite un bilan : le nombre de notes, la moyenne, la meilleure note, la plus faible, la mention obtenue, puis une ligne par note contenant autant d'étoiles que la partie entière de la note.

Le programme tient dans une seule classe. Les calculs sont réalisés par les méthodes `static` suivantes, appelées depuis `main` :

| Méthode à écrire                               | Rôle                                                                                                                                                                       |
|------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>boolean estValide(double note)</code>    | Retourne <code>true</code> si la note est comprise entre 0 et 20 inclus.                                                                                                   |
| <code>double moyenne(double[] notes)</code>    | Retourne la moyenne des notes du tableau.                                                                                                                                  |
| <code>double meilleure(double[] notes)</code>  | Retourne la plus grande note.                                                                                                                                              |
| <code>double plusFaible(double[] notes)</code> | Retourne la plus petite note.                                                                                                                                              |
| <code>String mention(double moyenne)</code>    | Retourne la mention : « Insuffisant » en dessous de 10, « Passable » à partir de 10, « Assez bien » à partir de 12, « Bien » à partir de 14, « Excellent » à partir de 16. |
| <code>String barre(double note)</code>         | Retourne une chaîne d'étoiles : une étoile par point entier de la note.                                                                                                    |

**Exemple d'exécution** (les valeurs saisies sont 3, 10, 25, 15 et 12.5) :

```
Nombre de notes : 3
Note 1 : 10
Note 2 : 25
Note invalide (0 à 20). Note 2 : 15
Note 3 : 12.5
```

```
Nombre de notes : 3
Moyenne : 12.5
Meilleure note : 15.0
Plus faible : 10.0
Mention : Assez bien
Note 1 | *****
Note 2 | *****
Note 3 | *****
```

**Astuce** : selon la configuration du poste, Scanner attend une virgule (12,5) ou un point (12.5). Pour imposer le point, créez-le ainsi : `new Scanner(System.in).useLocale(Locale.US)`.

### 1.3 Critères d'acceptation

Votre travail est accepté lorsque **tous** les critères suivants sont vérifiés.

| N°  | Critère                                                                                                                    | Vérification     |
|-----|----------------------------------------------------------------------------------------------------------------------------|------------------|
| CA1 | Le fichier <code>CarnetDeNotes.java</code> compile sans erreur et contient les six méthodes avec les signatures demandées. | Compilation      |
| CA2 | <code>estValide</code> accepte 0 et 20, et refuse -1 et 20.5.                                                              | Test automatique |
| CA3 | Une note invalide saisie au clavier est refusée, un message s'affiche et la même note est redemandée.                      | À la main        |
| CA4 | Pour les notes 10, 15 et 12.5, <code>moyenne</code> retourne 12.5.                                                         | Test automatique |
| CA5 | Pour les mêmes notes, <code>meilleure</code> retourne 15.0 et <code>plusFaible</code> retourne 10.0.                       | Test automatique |
| CA6 | <code>mention</code> retourne la bonne mention aux bornes : 9.99, 10, 12, 14 et 16.                                        | Test automatique |
| CA7 | <code>barre(12.5)</code> retourne 12 étoiles ; <code>barre(0)</code> retourne une chaîne vide.                             | Test automatique |
| CA8 | Avec les saisies de l'exemple, l'affichage est identique à l'exemple d'exécution.                                          | À la main        |
| CA9 | <code>main</code> ne refait aucun calcul : il appelle les méthodes. Les noms de variables sont explicites.                 | Relecture        |

## 1.4 Tester son code

Un développeur ne se contente pas de regarder si son programme « a l'air de marcher » : il le **teste**. La méthode tester ci-dessous compare le résultat obtenu au résultat attendu et affiche [OK] ou [ECHEC]. Chaque test porte le numéro du critère qu'il vérifie. Créez le fichier TestCarnetDeNotes.java dans le même dossier que CarnetDeNotes.java, puis exécutez-le.

```
public class TestCarnetDeNotes {
    static int reussis = 0, echoues = 0;

    // Compare le résultat obtenu au résultat attendu et affiche le verdict.
    static void tester(String nom, Object obtenu, Object attendu) {
        if (String.valueOf(obtenu).equals(String.valueOf(attendu))) {
            reussis++;
            System.out.println("[OK]      " + nom);
        } else {
            echoues++;
            System.out.println("[ECHEC] " + nom);
            System.out.println("    obtenu : " + obtenu + " | attendu : " + attendu);
        }
    }

    public static void main(String[] args) {
        double[] notes = {10, 15, 12.5};
        tester("CA2 estValide(-1)", CarnetDeNotes.estValide(-1), false);
        tester("CA2 estValide(20)", CarnetDeNotes.estValide(20), true);
        tester("CA2 estValide(20.5)", CarnetDeNotes.estValide(20.5), false);
        tester("CA4 moyenne", CarnetDeNotes.moyenne(notes), 12.5);
        tester("CA5 meilleure", CarnetDeNotes.meilleure(notes), 15.0);
        tester("CA5 plusFaible", CarnetDeNotes.plusFaible(notes), 10.0);
        tester("CA6 mention(9.99)", CarnetDeNotes.mention(9.99), "Insuffisant");
        tester("CA6 mention(10)", CarnetDeNotes.mention(10), "Passable");
        tester("CA6 mention(12)", CarnetDeNotes.mention(12), "Assez bien");
        tester("CA6 mention(14)", CarnetDeNotes.mention(14), "Bien");
        tester("CA6 mention(16)", CarnetDeNotes.mention(16), "Excellent");
        tester("CA7 barre(12.5)", CarnetDeNotes.barre(12.5), "*****");
        tester("CA7 barre(0)", CarnetDeNotes.barre(0), "");
        System.out.println(reussis + " test(s) réussi(s), " + echoues + " échec(s)");
    }
}
```

1. Tant qu'une méthode n'est pas encore écrite, mettez ses lignes de test en commentaire (//) pour que le fichier compile.
2. En cas d'[ECHEC], comparez les valeurs « obtenu » et « attendu », corrigez votre méthode, puis relancez les tests.
3. Ajoutez **au moins trois tests de votre choix** sur des cas limites : un tableau d'une seule note, des notes toutes égales, la note 20.

## Partie 2 : Classes et Objets

### 2.1 Principes à retenir

| Principe               | À retenir                                                                                                                                   |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Classe et objet</b> | La classe est le modèle ; l'objet est un exemplaire créé avec <code>new</code> .                                                            |
| <b>Encapsulation</b>   | Les attributs sont <code>private</code> ; on y accède par des méthodes publiques (accesseurs) qui contrôlent les valeurs.                   |
| <b>Constructeur</b>    | Méthode spéciale, du même nom que la classe, qui initialise l'objet. <code>this(...)</code> appelle un autre constructeur de la classe.     |
| <b>Membre static</b>   | Appartient à la classe et non à un objet : il est partagé par tous les objets.                                                              |
| <b>Référence</b>       | Une variable objet contient une référence : <code>Livre l2 = l1</code> ; ne copie pas le livre, les deux noms désignent le même objet.      |
| <b>Redéfinition</b>    | <code>toString()</code> définit le texte affiché par <code>System.out.println(objet)</code> ; on la redéfinit avec <code>@Override</code> . |

### 2.2 Exercice : la bibliothèque

Écrire un programme de gestion d'une petite bibliothèque. Un **livre** possède un titre, un auteur, un numéro ISBN (un entier) et un état : emprunté ou disponible. Un livre nouvellement créé est disponible. On peut l'emprunter s'il est disponible et le rendre s'il est emprunté ; sinon l'opération est refusée et un message s'affiche. Une **bibliothèque** contient un nombre limité de livres : on peut y ajouter un livre, rechercher un livre par son titre, compter les livres disponibles et tout afficher.

Le programme comporte trois classes : `Livre`, `Bibliotheque` et `Main`. Tous les attributs sont `private`.

| Classe Livre                                                                                            | Rôle                                                                                                                                       |
|---------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <code>Livre(String titre, String auteur, int isbn)</code>                                               | Crée un livre disponible avec les valeurs reçues.                                                                                          |
| <code>Livre()</code>                                                                                    | Appelle le constructeur précédent avec <code>this("Inconnu", "Inconnu", 0)</code> .                                                        |
| <code>getTitre()</code> , <code>getAuteur()</code> , <code>getIsbn()</code> , <code>isEmprunte()</code> | Retournent la valeur de l'attribut.                                                                                                        |
| <code>setTitre(String)</code> , <code>setAuteur(String)</code> , <code>setIsbn(int)</code>              | Modifient l'attribut. <code>setIsbn</code> ignore une valeur négative : l'ISBN reste inchangé.                                             |
| <code>boolean emprunter()</code>                                                                        | Si le livre est disponible : il devient emprunté, retourne <code>true</code> . Sinon : affiche un message, retourne <code>false</code> .   |
| <code>boolean rendre()</code>                                                                           | Si le livre est emprunté : il redevient disponible, retourne <code>true</code> . Sinon : affiche un message, retourne <code>false</code> . |
| <code>String toString()</code>                                                                          | Retourne « titre - auteur - ISBN numéro - état », l'état étant <code>disponible</code> ou <code>indisponible</code> .                      |
| <code>void afficherDetails()</code>                                                                     | Affiche le texte retourné par <code>toString()</code> .                                                                                    |
| <code>static int getNbLivres()</code>                                                                   | Retourne le nombre total de livres créés depuis le début du programme.                                                                     |

| Classe Bibliotheque                    | Rôle                                                                             |
|----------------------------------------|----------------------------------------------------------------------------------|
| Bibliotheque(int capacite)             | Crée une bibliothèque vide pouvant contenir capacite livres (tableau de Livre).  |
| boolean ajouterLivre(Livre l)          | Ajoute le livre et retourne true ; retourne false si la bibliothèque est pleine. |
| Livre rechercherParTitre(String titre) | Retourne le livre portant ce titre, ou null s'il n'existe pas.                   |
| int nombreDisponibles()                | Retourne le nombre de livres non empruntés.                                      |
| void afficherTout()                    | Affiche les détails de chaque livre.                                             |

**Classe Main et exemple d'exécution.** Créez une bibliothèque de 10 places, ajoutez-y le livre « Germinal » de Zola (ISBN 123) et un livre créé avec le constructeur par défaut, empruntez le premier, puis affichez tout ainsi que le nombre de livres disponibles :

```
Germinal - Zola - ISBN 123 - indisponible
Inconnu - Inconnu - ISBN 0 - disponible
Livres disponibles : 1
```

## 2.3 Critères d'acceptation

Votre travail est accepté lorsque **tous** les critères suivants sont vérifiés.

| N°   | Critère                                                                                                                                     | Vérification           |
|------|---------------------------------------------------------------------------------------------------------------------------------------------|------------------------|
| CA1  | Les trois fichiers compilent sans erreur ; tous les attributs sont private.                                                                 | Compilation, relecture |
| CA2  | Le constructeur avec paramètres enregistre le titre, l'auteur et l'ISBN ; le livre est disponible.                                          | Test automatique       |
| CA3  | Le constructeur par défaut donne « Inconnu », « Inconnu », 0 en appelant this(...).                                                         | Test automatique       |
| CA4  | setIsbn refuse une valeur négative : l'ISBN reste inchangé.                                                                                 | Test automatique       |
| CA5  | emprunter() retourne true la première fois et false si le livre est déjà emprunté.                                                          | Test automatique       |
| CA6  | rendre() retourne true si le livre était emprunté et false sinon.                                                                           | Test automatique       |
| CA7  | toString() respecte exactement le format demandé.                                                                                           | Test automatique       |
| CA8  | getNbLivres() augmente de 1 à chaque livre créé, quel que soit le constructeur utilisé.                                                     | Test automatique       |
| CA9  | ajouterLivre refuse un livre quand la bibliothèque est pleine ; rechercherParTitre retourne le livre ou null ; nombreDisponibles est exact. | Test automatique       |
| CA10 | La classe Main produit exactement l'affichage de l'exemple d'exécution.                                                                     | À la main              |
| CA11 | Après Livre l2 = l1; puis l2.setTitre("Nana");, vous savez dire ce qu'affiche l1.getTitre() et expliquer pourquoi.                          | Explication écrite     |

## 2.4 Tester son code

Créez le fichier `TestBibliotheque.java` dans le même dossier que vos classes, puis exécutez-le. Comme dans la Partie 1, chaque test porte le numéro du critère qu'il vérifie.

```
public class TestBibliotheque {
    // Recopiez ici les deux compteurs et la méthode tester(...) de TestCarnetDeNotes.

    public static void main(String[] args) {
        int avant = Livre.getNbLivres();
        Livre l1 = new Livre("Germinal", "Zola", 123);
        Livre l0 = new Livre();
        tester("CA2 titre", l1.getTitre(), "Germinal");
        tester("CA2 isbn", l1.getIsbn(), 123);
        tester("CA2 disponible", l1.isEmprunte(), false);
        tester("CA3 default", l0.getTitre() + "/" + l0.getIsbn(), "Inconnu/0");
        l1.setIsbn(-5);
        tester("CA4 isbn negatif refuse", l1.getIsbn(), 123);
        tester("CA5 premier emprunt", l1.emprunter(), true);
        tester("CA5 second emprunt", l1.emprunter(), false);
        tester("CA6 retour", l1.rendre(), true);
        tester("CA6 second retour", l1.rendre(), false);
        tester("CA7 toString", l1, "Germinal - Zola - ISBN 123 - disponible");
        tester("CA8 compteur", Livre.getNbLivres() - avant, 2);

        Bibliotheque b = new Bibliotheque(1);
        tester("CA9 ajout", b.ajouterLivre(l1), true);
        tester("CA9 bibliotheque pleine", b.ajouterLivre(l0), false);
        tester("CA9 recherche", b.rechercherParTitre("Germinal"), l1);
        tester("CA9 titre absent", b.rechercherParTitre("Nana"), null);
        tester("CA9 disponibles", b.nombreDisponibles(), 1);
        System.out.println(reussis + " test(s) réussi(s), " + echoues + " échec(s)");
    }
}
```

1. En cas d' [ECHEC], relisez le critère concerné, corrigez votre classe, puis relancez **tous** les tests.
2. Ajoutez **au moins trois tests de votre choix** : par exemple un ISBN égal à 0, la recherche dans une bibliothèque vide, le nombre de livres disponibles après un emprunt.

## **Partie 3 : Prompt Engineering – Développer avec un assistant IA**

En entreprise, les assistants IA (ChatGPT, Claude, Gemini, Copilot...) font désormais partie des outils du développeur. La qualité du résultat dépend directement de la qualité de la demande (le *prompt*), et **le développeur reste responsable du code qu'il livre**. Cette partie se traite après les Parties 1 et 2.

### **3.1 Le modèle de prompt professionnel**

Un prompt professionnel se rédige comme une spécification : court, structuré et vérifiable. Utilisez systématiquement les six rubriques suivantes.

| Rubrique                      | Contenu                                                                            |
|-------------------------------|------------------------------------------------------------------------------------|
| <b>Rôle</b>                   | Le profil attendu de l'IA : « Tu es un développeur Java senior. »                  |
| <b>Contexte</b>               | Le projet, la version de Java, le niveau de l'équipe, le code existant.            |
| <b>Tâche</b>                  | Une seule action précise : générer, corriger, relire, tester ou documenter.        |
| <b>Contraintes</b>            | Ce qui est imposé ou interdit : conventions de nommage, bibliothèques, périmètre.  |
| <b>Format de sortie</b>       | La forme de la réponse : code seul, tableau, liste classée, longueur maximale.     |
| <b>Critères d'acceptation</b> | Ce qui permet de dire que la réponse est correcte : cas de test, résultat attendu. |

#### **Exemple :**

|             |                                                                            |
|-------------|----------------------------------------------------------------------------|
| RÔLE        | : Tu es un développeur Java senior.                                        |
| CONTEXTE    | : Application console de gestion de bibliothèque, Java 25, sans framework. |
| TÂCHE       | : Relis la classe Livre ci-dessous et signale les défauts.                 |
| CONTRAINTES | : Ne réécris pas la classe. Respecte l'énoncé fourni (isbn est un entier). |
| FORMAT      | : Tableau : défaut   gravité (bloquant, majeur, mineur)   correction.      |
| CRITÈRES    | : 5 défauts au maximum, classés du plus grave au moins grave.              |
| [énoncé]    | [code]                                                                     |

### **3.2 Travail demandé**

Pour chaque mission, rédigez votre prompt avec le modèle 3.1, testez la réponse obtenue, puis améliorez le prompt si le résultat n'est pas conforme (version 1, puis version 2).

**Mission 1 – Faire relire son code.** Soumettez **votre** classe Livre (Partie 2) à une revue de code en adaptant l'exemple 3.1. Pour chaque défaut signalé, indiquez si vous l'acceptez ou le refusez, et pourquoi.

**Mission 2 – Générer des cas de test.** Demandez à l'IA un tableau de cas de test pour les méthodes emprunter() et rendre() (état initial, action, résultat attendu). Exécutez ces cas dans votre classe Main et signalez tout écart.

**Mission 3 – Corriger une anomalie.** Le programme suivant doit afficher le minimum, le maximum et la somme d'une série d'entiers terminée par 0 (le 0 ne compte pas). Il compile, mais ses résultats sont faux.

```

import java.util.Scanner;

public class MinMaxSomme {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int min = 0, max = 0, somme = 0;
        int v;
        do {
            System.out.print("Valeur (0 pour arrêter) : ");
            v = sc.nextInt();
            if (v > max) max = v;
            if (v < min) min = v;
            somme = v;
        } while (v != 0);
        System.out.println("Max = " + max + ", Min = " + min);
        System.out.println("Somme = " + somme);
    }
}
    
```

1. Exécutez le programme avec les jeux d'essai du tableau et notez les résultats obtenus.
2. Rédigez un prompt qui fournit le code, le résultat **obtenu** et le résultat **attendu**, et qui demande la cause de chaque anomalie avant toute correction.
3. Corrigez le code, puis rejouez les trois jeux d'essai pour valider.

| Valeurs saisies            | Résultat attendu                 | Avant correction | Après correction |
|----------------------------|----------------------------------|------------------|------------------|
| 5, 3, 8, 0                 | Max = 8, Min = 3, Somme = 16     |                  |                  |
| -4, -2, -9, 0              | Max = -2, Min = -9, Somme = -15  |                  |                  |
| 0 (dès la première saisie) | Message « aucune valeur saisie » |                  |                  |

**Mission 4 – Auditer un code généré par l’IA.** Un collègue a obtenu d’un assistant IA la classe Livre suivante et s’apprête à l’intégrer telle quelle. Elle compile sans erreur et semble propre ; **elle contient pourtant au moins trois défauts** par rapport à l’énoncé de la section 2.2.

```
public class Livre {
    private String titre, auteur;
    private int isbn;
    private boolean emprunte;
    private static int nbLivres = 0;

    public Livre() { this("Inconnu", "Inconnu", 0); }

    public Livre(String titre, String auteur, int isbn) {
        this.titre = titre;
        this.auteur = auteur;
        isbn = isbn;
        this.emprunte = false;
        nbLivres++;
    }

    public String getTitre() { return titre; }
    public String getAuteur() { return auteur; }
    public int getIsbn() { return isbn; }
    public boolean isEmprunte() { return emprunte; }
    public static int getNbLivres() { return nbLivres; }

    public void setTitre(String titre) { this.titre = titre; }
    public void setAuteur(String auteur) { this.auteur = auteur; }
    public void setIsbn(int isbn) { this.isbn = isbn; }

    public boolean emprunter() {
        if (emprunte) {
            emprunte = true;
            return true;
        }
        System.out.println("Le livre n'est pas disponible.");
        return false;
    }

    public boolean rendre() {
        emprunte = false;
        return true;
    }

    @Override
    public String toString() {
        return titre + " - " + auteur + " - ISBN " + isbn + " - "
            + (emprunte ? "indisponible" : "disponible");
    }

    public void afficherDetails() { System.out.println(this); }
}
```

1. **Sans exécuter le code**, relisez-le en le comparant à l’énoncé 2.2 et notez les défauts que vous soupçonnez.

2. Remplacez provisoirement votre classe Livre par celle-ci et lancez TestBibliotheque : quels critères d'acceptation échouent ?
3. Complétez le tableau, puis corrigez la classe. Ne validez le code que lorsque tous les tests passent.

| Défaut constaté | Méthode concernée | Critère non respecté | Correction |
|-----------------|-------------------|----------------------|------------|
|                 |                   |                      |            |
|                 |                   |                      |            |
|                 |                   |                      |            |
|                 |                   |                      |            |

**À retenir :** un code qui compile et qui « a l'air correct » n'est pas un code validé. Seuls la relecture face à l'énoncé et les tests permettent d'accepter un code généré.

### 3.3 Livrable : fiche de prompts

Pour les missions 1 à 3, consignez la première et la dernière version de votre prompt :

| Mission | Version | Prompt | Résultat conforme ? | Ce qui a été modifié |
|---------|---------|--------|---------------------|----------------------|
| 1       | v1      |        |                     |                      |
| 1       | v2      |        |                     |                      |
| ...     |         |        |                     |                      |

### 3.4 Règles professionnelles

- **Confidentialité :** ne jamais transmettre de données clients, de mots de passe, de clés d'accès ou de code propriétaire à un outil non validé par l'entreprise.
- **Vérification :** tout code généré est compilé, testé et relu avant d'être intégré ; une réponse peut être fausse tout en paraissant sûre.
- **Responsabilité :** l'auteur du commit répond du code, qu'il ait été écrit à la main ou généré.
- **Traçabilité :** conserver les prompts utiles et signaler l'usage de l'IA dans le compte rendu.